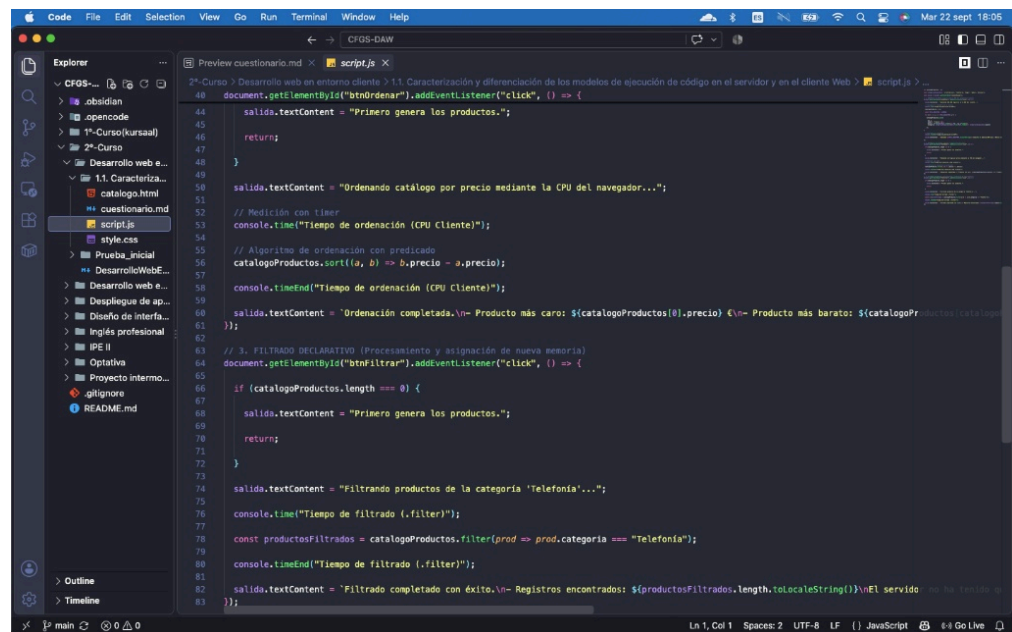


Actividad Práctica Guiada: Tiempos de Ejecución en el Navegador

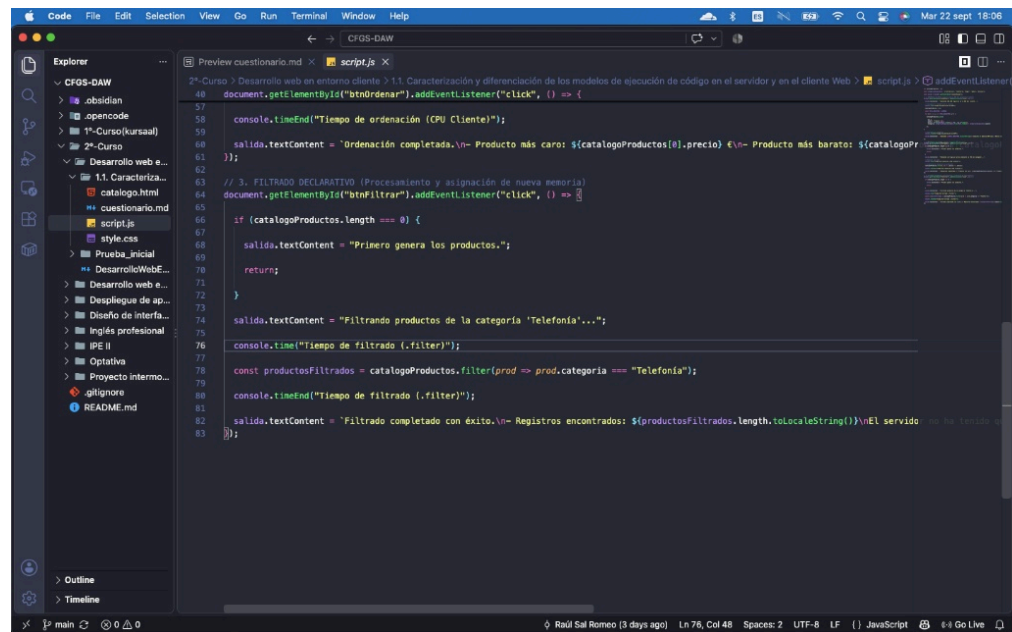
Modificaciones Guiadas

- **Modificación 1 (Invertir el criterio de ordenación):**
 - **Instrucción:** Localiza la función del método `.sort()` en `script.js`. Modifícala para que ordene los artículos de mayor a menor precio (orden descendente). Ajusta el mensaje de salida en visor para que muestre primero el artículo más caro y luego el más barato.
 - **Código Implementado:**



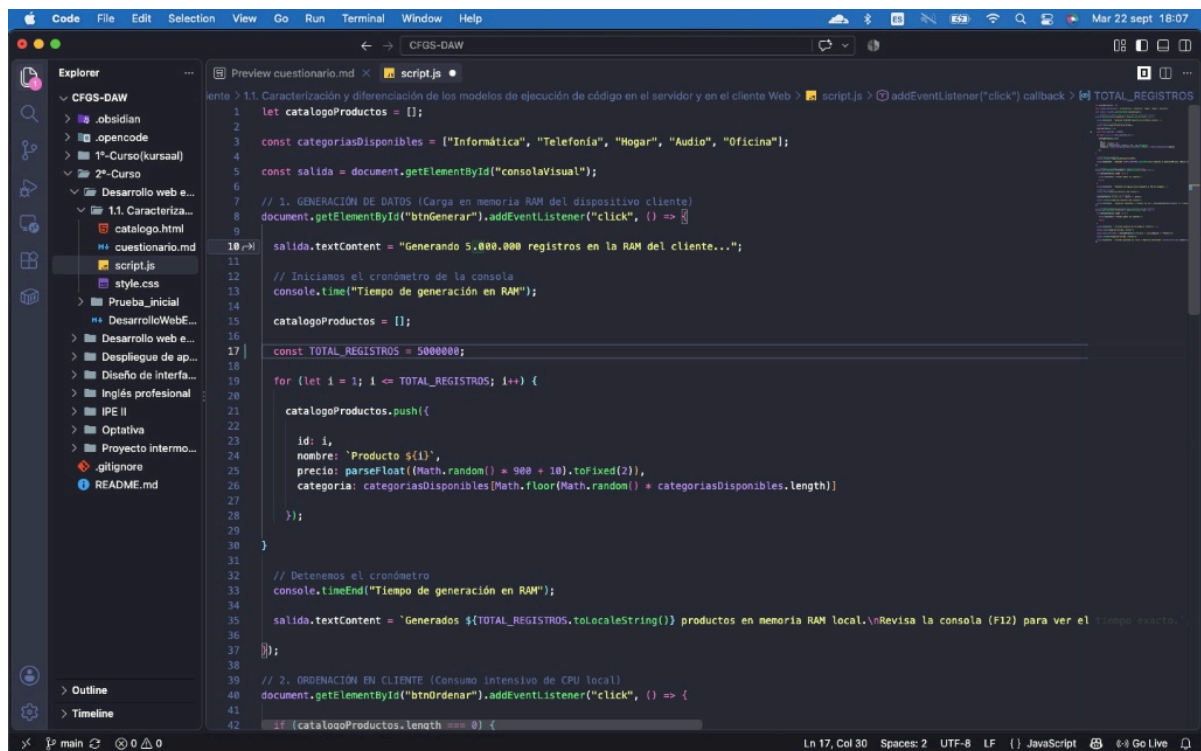
```
44     salida.textContent = "Primero genera los productos.";
45     return;
46 }
47
48 salida.textContent = "Ordenando catálogo por precio mediante la CPU del navegador...";
49
50 // Medición con timer
51 console.time("Tiempo de ordenación (CPU Cliente)");
52
53 // Algoritmo de ordenación con predicado
54 catalogoProductos.sort((a, b) => b.precio - a.precio);
55
56 console.timeEnd("Tiempo de ordenación (CPU Cliente)");
57
58 salida.textContent = "Ordenación completada.\n- Producto más caro: ${catalogoProductos[0].precio} €\n- Producto más barato: ${catalogoProductos[catalogoProductos.length - 1].precio} €";
59
60 // 3. FILTRADO DECLARATIVO (Procesamiento y asignación de nueva memoria)
61 document.getElementById("btnFiltrar").addEventListener("click", () => {
62     if (catalogoProductos.length === 0) {
63         salida.textContent = "Primero genera los productos.";
64         return;
65     }
66     salida.textContent = "Filtrando productos de la categoría 'Telefonía'...";
67     console.time("Tiempo de filtrado (.filter())");
68     const productosFiltrados = catalogoProductos.filter(prod => prod.categoria === "Telefonía");
69     console.timeEnd("Tiempo de filtrado (.filter())");
70     salida.textContent = "Filtrado completado con éxito.\n- Registros encontrados: ${productosFiltrados.length.toLocaleString()}.\nEl servidor no ha tenido que procesar nada.";
71 });
```

- **Modificación 2 (Cambiar la categoría del filtro):**
 - **Instrucción:** Modifica el predicado del método `.filter()` para que seleccione únicamente los productos de la categoría "Telefonía" en lugar de "Informática".
 - **Código Implementado:**



- **Modificación 3 (Aumento de volumen y prueba de estrés):**

- **Instrucción:** Localiza la variable constante TOTAL y sustituye el valor de 150000 por 500000 registros (o 5.000.000). Vuelve a pulsar los botones en orden y anota los nuevos tiempos mostrados en la consola.
- **Código Implementado:**



Cuestión 1:

- Tiempos registrados:
 - Con 150.000 elementos→aprox 36.77ms
 - Con 500.000 elementos→aprox 38.23ms
- Tipo de crecimiento: Crece de forma superlineal ($n \cdot \log n$). No es lineal porque, al haber más datos, el algoritmo necesita hacer muchas más comparaciones por cada elemento.

Cuestión 2:

- Tiempo total acumulado:
 $10.000 \text{ usuarios} * 300 \text{ ms} = 3.000.000 \text{ ms} = 3.000 \text{ s} \approx 50 \text{ minutos de tiempo de CPU.}$
- Beneficio para la empresa:
 - Esos 50 minutos se reparten en paralelo entre los móviles y ordenadores de los usuarios; nadie espera colas y la respuesta es casi inmediata (aprox 300ms).
 - El servidor y la base de datos no se saturan ni se caen al ahorrarse 10.000 consultas ORDER BY continuas.

Cuestión 4:

- ¿Es viable? No.
 - Ocuparía más de 1 o 2 GB de memoria RAM en el navegador, provocando que la pestaña se cierre o falle[cite: 4, 5].
 - Descargar tantos datos tardaría demasiado y consumiría muchos datos.
 - Congelaría la pantalla del usuario al intentar ordenar tantos elementos.
- Solución propuesta: Usar paginación en el servidor[cite: 4, 5]. La base de datos envía los datos en paquetes pequeños (por ejemplo, de 50 en 50 con LIMIT y OFFSET) según el usuario vaya haciendo scroll o cambiando de página.